**A CASE-STUDY BASED REPORT**

**ON**

# Hospital Readmission Prediction using Elastic Net Regularized Logistic Regression

***submitted in partial fulfilment of the requirements for the Course***

## MACHINE LEARNING

## BACHELOR OF TECHNOLOGY

**in**

**A.Y.: 2026-27**

**by**

**Krishnasree** **2520080009**

**Vennela** **2520080018**

***Under the Esteemed guidance of***

Mrs.ch.Anitha

Asst.Professor, Dept. of CS&IT

**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

# INDEX

Abstract
1. Introduction
    1.1 Background
    1.2 Motivation for the Project
    1.3 Overview of the Work Carried Out
    1.4 Scope and Responsible-Use Statement
2. Problem Statement
    2.1 Case-Study Scenario
    2.2 Central Case-Study Question
    2.3 Formal Problem Definition
    2.4 Objectives that Guide the Study
    2.5 Assumptions and Limitations
3. Dataset and Exploratory Data Analysis
    3.1 About the Dataset
    3.2 Purpose of EDA
    3.3 Preparation Steps Actually Performed
    3.4 EDA Exploration – Implementation
    3.5 Summary of EDA Findings
4. Algorithm Selection and Implementation
    4.1 System Architecture and Workflow
    4.2 Feature Description
    4.3 Model and Preprocessing Pipeline
    4.4 Prediction and Risk Categorisation
    4.5 Application Interface and Data Storage
    4.6 Testing and Validation Plan
    4.7 Limitations and Future Enhancements
5. Conclusion
References

# ABSTRACT

Hospital readmission refers to a patient returning to a hospital after discharge. Avoidable readmissions can increase healthcare costs and may indicate that a patient needs closer follow-up or additional support. This case study presents RE:ADMIT, a machine-learning-based educational prototype that estimates readmission risk from selected patient and discharge-related details. The application accepts fields such as age, length of stay, previous admissions, emergency visits, medication count, comorbidity count, selected diagnosis, discharge destination, abnormal-lab indicator, follow-up planning, medication concerns, and mobility concerns.

The prototype uses a scikit-learn preprocessing pipeline with a Random Forest classifier and exposes prediction functionality through a Flask web application. Assessment records are stored locally in an SQLite database, and the interface supports patient assessment and CSV-based bulk processing. The supplied code generates a synthetic demonstration dataset when a compatible saved model is unavailable. Therefore, its predictions are intended to demonstrate the software workflow rather than provide clinically reliable estimates. No real-world performance metrics are reported because a real, licensed dataset and an independent evaluation were not supplied. The report explains the case-study problem, dataset design, exploratory analysis approach, algorithm, implementation, limitations, and responsible-use requirements.

Keywords: hospital readmission, machine learning, Random Forest, Flask, patient data, risk prediction, synthetic dataset, healthcare analytics.

# 1. INTRODUCTION

## 1.1 Background

Hospitals collect many kinds of information during a patient's stay. Examples include demographic details, diagnosis, duration of hospitalisation, previous admissions, medicines, test indicators, mobility needs, and discharge plans. Reviewing these details together can be time-consuming, especially when many records need to be examined. Machine learning can be used to identify patterns in structured data and produce an estimated probability for a defined outcome.

RE:ADMIT is a prototype designed to demonstrate how a prediction model can be connected to a simple web interface. A user enters a patient or study ID and the available patient-related features. The application passes the values to a trained or demonstration model, receives a probability estimate, converts it to a display category, and stores the assessment for later review.

## 1.2 Motivation for the Project

The project is motivated by the need to organise patient information and demonstrate a repeatable way to review potential readmission risk. A structured prediction workflow can help students understand the complete machine-learning process, from feature preparation and model inference to web integration and record storage. In a real healthcare setting, a properly validated model might support a care team's review, but it must not replace professional judgement.

## 1.3 Overview of the Work Carried Out

- Designed a web-based patient assessment workflow under the project name RE:ADMIT.
- Defined numeric, categorical, and indicator features used by the prediction pipeline.
- Implemented a preprocessing stage using a ColumnTransformer and OneHotEncoder for categorical fields.

- Used a Random Forest classifier within a scikit-learn Pipeline to produce a probability estimate.
- Connected the model to a Flask application and enabled cross-origin requests through Flask-CORS.
- Added local SQLite persistence for assessment details, prediction category, risk percentage, review factors, model version, and timestamp.
- Provided a sample CSV format to demonstrate bulk assessment input.

### 1.4 Scope and Responsible-Use Statement

The scope of this case study is an educational prototype for structured-data readmission-risk prediction. It covers input validation, model inference, a browser-based interface, local record storage, and a planned evaluation process. The current demonstration model is trained on generated synthetic records rather than real hospital outcomes. It has not been clinically validated, externally tested, calibrated for a particular hospital, or approved for medical use.

Only fictional or de-identified demonstration records should be used with the prototype. It does not include production-grade authentication, role-based access, encryption, audit logging, or institutional privacy controls. The displayed category is a demonstration threshold and must not be used to make diagnosis, treatment, discharge, or patient-care decisions.

## 2. PROBLEM STATEMENT

### 2.1 Case-Study Scenario

Consider a hospital team that wants to review patient records after discharge and identify which records may require a closer follow-up review. Manually comparing several features across many records can be difficult. The case study explores whether a machine-learning workflow can take a consistent set of structured inputs and return an estimated readmission probability with a simple risk label.

### 2.2 Central Case-Study Question

How can a machine-learning model be integrated into a web-based application to estimate hospital readmission risk from selected patient and discharge-related features, while clearly communicating the limitations of the estimate?

### 2.3 Formal Problem Definition

Let each patient record be represented by a feature vector x containing demographic, admission-history, clinical-summary, and discharge-planning fields. The target y is a binary indicator representing whether a readmission outcome is present in the data used to train the model. A classifier estimates $P(y = 1 \mid x)$, the probability of the defined outcome given the input features. In this prototype, the model returns a probability which is displayed as a percentage. The target definition and synthetic label-generation process are part of the demonstration and are not equivalent to verified clinical outcomes.

### 2.4 Objectives that Guide the Study

- Build a clear interface for entering a consistent set of patient-related features.
- Validate required fields and accepted categories before model inference.
- Apply the same preprocessing steps to incoming records as those used during model training.
- Generate a probability estimate and display it in an understandable format.
- Store each assessment with its timestamp and model-version label for demonstration and review.

- Support CSV input so that multiple fictional records can be processed efficiently.
- Document limitations and responsible-use safeguards, especially the difference between synthetic demonstrations and clinical evidence.

### 2.5 Assumptions and Limitations

- The input fields are assumed to be entered correctly and to follow the expected format.
- The application expects a binary classifier that supports predict_proba(), with class index 1 representing the readmission outcome.
- If a compatible model file is absent, the code creates a demonstration model using generated synthetic data.
- The synthetic outcome is generated from a programmed probability rule; it is not a record of actual patient outcomes.
- The Low, Moderate, and High labels use illustrative thresholds of below 30%, 30% to below 60%, and 60% or above. These cut-offs are not clinically validated.
- No verified accuracy, precision, recall, F1-score, AUROC, calibration result, or external validation result is available in the provided project materials.
- The prototype has no production authentication or comprehensive security controls and should not store identifiable patient information.

## 3. DATASET AND EXPLORATORY DATA ANALYSIS

### 3.1 About the Dataset

The application code includes a reproducible synthetic-data generator. When a compatible file named hospital_readmission_model.pkl is not available, it generates 5,000 artificial records for model training. The generated records include numeric fields such as age, length of stay, prior admissions, emergency visits, medication count, comorbidity count, and several binary indicators. Categorical fields include sex, diagnosis, and discharge destination.

| Feature group | Fields represented | Data type |
|---|---|---|
| Demographic | age, sex | Numeric / categorical |
| Admission history | length_of_stay, prior_admissions, emergency_visits | Numeric counts |
| Medication and conditions | medication_count, comorbidity_count | Numeric counts |
| Review indicators | lab_abnormal, followup_planned, medication_concern, mobility_concern | Binary indicators |
| Clinical category | diagnosis | Categorical |
| Discharge plan | discharge | Categorical |
| Target | synthetic binary readmission outcome | Binary |

The synthetic generator samples feature values from programmed distributions and creates a probability score from a predefined mathematical rule with added random variation. A binary outcome is then sampled from that probability. This setup is useful for checking whether the application runs end to end, but it can create overly simplified relationships that do not represent real hospital populations.

The project also includes a small CSV example for trying the interface. It contains three fictional demonstration rows. That example is for checking data entry and CSV processing; it should not be treated as the training dataset or as clinical evidence.

## 3.2 Purpose of EDA

Exploratory Data Analysis (EDA) is normally used to understand a dataset before training a model. It helps identify missing values, invalid values, unusual ranges, class imbalance, feature distributions, and relationships between variables and the target. For a real readmission study, EDA would also check whether records are duplicated, whether the outcome window is consistently defined, and whether data leakage or subgroup imbalance may affect results.

## 3.3 Preparation Steps Actually Performed

The supplied application code performs the following data-generation and model-preparation steps when it needs to build its demonstration model:

- Creates a Pandas DataFrame with 5,000 synthetic records using a fixed random seed for reproducibility.
- Generates bounded numeric values for age, length of stay, previous admissions, emergency visits, medication count, and comorbidity count.
- Generates binary indicators for abnormal labs, follow-up planning, medication concerns, and mobility concerns.
- Generates categorical values for sex, diagnosis, and discharge destination.
- Constructs a synthetic outcome probability using a programmed formula and samples a binary target from it.
- Separates the numeric and categorical feature lists; numeric features pass through while categorical features are one-hot encoded.
- Fits the preprocessing and classifier steps together in a scikit-learn Pipeline and saves the resulting model using joblib.

The project files provided do not include a separate EDA notebook, plotted distribution charts, missing-value report, or a real-world dataset description. Therefore, this report does not claim that such analyses were completed. The checks below are recommended before using any real dataset.

## 3.4 EDA Exploration – Implementation

A complete EDA for a validated dataset should be performed before training. The following table records the checks that are relevant to this project and distinguishes the code-supported facts from analyses that still need to be run on a real dataset.

| EDA check | Why it matters | Status in supplied project |
|---|---|---|
| Dataset dimensions and data types | Confirms row count, columns, and expected types. | Synthetic generator specifies 5,000 rows; no standalone EDA report supplied. |
| Missing and invalid values | Prevents unreliable or failed predictions. | Input validation exists in the app; full dataset missingness report not supplied. |
| Numeric distributions | Shows typical values, spread, and outliers. | Not documented as plots or summary tables. |
| Categorical frequencies | Checks categories and uncommon values. | Allowed categories are defined in code; frequency analysis not supplied. |
| Target class balance | Shows whether readmission and non-readmission labels are imbalanced. | Synthetic target is generated; class counts are not reported. |
| Feature-target relationships | Helps identify patterns and possible leakage. | Synthetic outcome rule is programmed; independent relationship analysis not supplied. |
| Train/test evaluation | Estimates generalisation to unseen records. | No trustworthy held-out performance metrics supplied. |

### 3.5 Summary of EDA Findings

The main finding supported by the code is that the prototype has a defined synthetic-data pipeline with numeric and categorical fields and a generated binary outcome. The code uses a fixed random seed for feature generation and model fitting settings, which supports reproducibility of the demonstration process. However, exact class counts, feature summaries, correlation values, and model evaluation scores have not been documented in the supplied materials and are therefore not reported here.

For a stronger study, the next step is to obtain a suitable licensed dataset with a clear readmission definition and follow-up window, inspect it with Pandas, document missingness and distributions, split the data by patient where appropriate, and evaluate the final model on records not used during training.

## 4. ALGORITHM SELECTION AND IMPLEMENTATION

### 4.1 System Architecture and Workflow

The system follows a web-application workflow. A user enters patient information in the browser interface. The frontend sends the information to the Flask API as a request. The backend validates and normalises the input, creates a one-row Pandas DataFrame in the expected feature order, and passes it to the loaded scikit-learn pipeline. The model returns a probability estimate. The backend converts the estimate into a percentage and an illustrative category, prepares review flags from selected input fields, and saves the assessment to SQLite.

| Component | Responsibility |
|---|---|
| Browser interface | Collects fields, shows prediction results, supports assessment history and CSV workflows. |
| Flask API | Receives requests, validates input, coordinates prediction, and returns JSON responses. |
| Pandas | Represents a submitted record in tabular form for model inference. |
| scikit-learn Pipeline | Applies preprocessing and runs the Random Forest classifier. |
| Joblib | Loads and saves the trained model object. |
| SQLite database | Stores assessment fields, risk output, factors, model version, and timestamp. |
| CSV handling | Allows sample/bulk input and export of assessment records. |

### 4.2 Feature Description

| Feature | Description |
|---|---|
| age | Patient age in years; the current backend accepts 18–110. |
| length_of_stay | Number of days in the hospital stay. |
| prior_admissions | Number of previous admissions recorded. |
| emergency_visits | Number of emergency visits recorded. |
| medication_count | Count of medications recorded. |
| comorbidity_count | Count of additional conditions recorded. |
| lab_abnormal | Binary indicator for an abnormal-lab concern. |
| followup_planned | Binary indicator showing whether follow-up is planned. |
| medication_concern | Binary indicator for a medication-related concern. |
| mobility_concern | Binary indicator for mobility or self-care concern. |

| | |
|---|---|
| sex | Categorical value: Female, Male, Other, or Unknown. |
| diagnosis | Primary diagnosis category accepted by the prototype. |
| discharge | Discharge destination: home, home care, or facility. |

The project code validates several integer ranges and normalises diagnosis and discharge categories. It also limits the length of the patient/study ID and notes field. These checks help avoid malformed requests, but they do not establish that the clinical information itself is accurate.

## 4.3 Model and Preprocessing Pipeline

The selected classifier is Random Forest. It combines the predictions of multiple decision trees and can model non-linear patterns among structured features. The supplied code configures 160 trees, a maximum tree depth of 9, a minimum of 5 samples per leaf, class_weight='balanced', and random_state=42. These are the settings in the demonstration implementation, not evidence that this configuration is best for a real clinical dataset.

The preprocessing step is implemented with ColumnTransformer. Numeric features are passed through unchanged, while OneHotEncoder(handle_unknown='ignore') transforms categorical fields into numeric indicators and safely handles categories not observed during training. Both preprocessing and classification are stored in one Pipeline so that incoming records receive the same transformations used during fitting.

## 4.4 Prediction and Risk Categorisation

1. The application receives the submitted patient or study ID and feature values.
2. The backend validates required values and checks accepted diagnosis and discharge categories.
3. The feature values are arranged in a single-row Pandas DataFrame using the model's expected feature names.
4. The pipeline calls predict_proba() and obtains the estimated probability for class index 1.
5. The probability is converted to a percentage and displayed as a risk estimate.
6. The interface uses illustrative thresholds: Low below 30%, Moderate from 30% to below 60%, and High at 60% or above.
7. The application creates review flags from selected input fields, then stores the assessment and timestamp in SQLite.

The category thresholds are display rules chosen for the prototype. They have not been calibrated against clinical outcomes, and a percentage from the synthetic model must not be interpreted as a validated personal medical probability.

## 4.5 Application Interface and Data Storage

The frontend provides a patient assessment form and a dashboard-style presentation of results. The backend is served through Flask, and the project README instructs the user to run app.py and open the local address http://127.0.0.1:5000. The model is loaded from hospital_readmission_model.pkl if a compatible model is available. Otherwise, the application generates and saves the synthetic demonstration model.

Assessment records are stored in a local SQLite database named readmit_records.db by default. The stored information includes a unique assessment ID, patient/study ID, input fields, risk percentage, category, review factors, model version, and creation timestamp. The CSV feature provides a practical way to demonstrate multiple records without manually entering each record.

The current implementation is intended for local educational use. It does not include login, access control, encryption, or audit logging. Use fictional data only unless the application is reviewed and secured for an approved environment.

### 4.6 Testing and Validation Plan

Functional testing should verify that the application starts, required fields are checked, invalid categories are rejected, predictions are returned, and records are saved and retrieved correctly. CSV tests should include a valid file, a file with missing required columns, malformed numeric values, and rows with unsupported categories. The application should also be checked with boundary values such as minimum and maximum accepted ages and lengths of stay.

Model evaluation must be conducted separately from software testing. A real study should use a documented target definition and a patient-level train/validation/test split, and should report the class distribution, confusion matrix, sensitivity/recall, specificity, precision, F1-score, AUROC, calibration, and confidence intervals where appropriate. If the model will be considered for clinical use, external validation and fairness analysis across relevant patient groups are essential. These metrics are not included because no real outcome dataset or completed evaluation results were provided.

### 4.7 Limitations and Future Enhancements

- Replace synthetic training data with an appropriate licensed dataset containing verified outcomes and a clearly defined readmission window.
- Run and document exploratory data analysis, data cleaning, class-balance checks, and leakage checks.
- Compare Random Forest with suitable baseline models such as logistic regression and decision trees.
- Perform stratified or time-aware validation as appropriate, plus external validation on a separate setting.
- Assess probability calibration, subgroup performance, and the consequences of false positives and false negatives.
- Add authentication, role-based access, secure storage, encryption, audit trails, and a documented retention policy before handling sensitive information.
- Improve the interface with clearer explanations of model limitations and allow authorised reviewers to record follow-up decisions.
- Monitor model drift and retrain only under a documented governance process if a real deployment is ever approved.

## 5. CONCLUSION

RE:ADMIT demonstrates how a machine-learning model can be connected to a web application for a structured hospital readmission-risk assessment workflow. The prototype accepts patient-related inputs, validates them, applies a preprocessing pipeline, produces a probability estimate, displays an illustrative risk category, identifies selected review flags, and stores assessment records in SQLite. CSV support also demonstrates how multiple records can be handled through the same workflow.

The case study shows the practical connection between data preparation, a Random Forest classifier, a Flask API, and a browser-based interface. At the same time, the current model is a demonstration system because its fallback training process uses synthetic data and a programmed outcome rule. Its output is not clinically validated, and no reliable performance metrics are available in the supplied project files. The results must not be used to guide diagnosis, treatment, discharge, or patient care.

The most important next step is to evaluate the approach with an appropriate real-world dataset, conduct documented EDA and independent testing, and introduce robust privacy and security controls. With those improvements and suitable clinical oversight, the project could serve as a foundation for studying responsible data-driven support for post-discharge follow-up.

# REFERENCES

1. Project source code: RE:ADMIT working readmission-risk prototype, app.py (Flask, Pandas, scikit-learn, Random Forest, SQLite, and joblib implementation supplied with the project).

2. Project documentation: RE:ADMIT — Working readmission-risk prototype, README.md (setup instructions, feature description, CSV format, and model limitations).

3. scikit-learn documentation. “RandomForestClassifier.” https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

4. scikit-learn documentation. “Pipeline.” https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

5. scikit-learn documentation. “ColumnTransformer.” https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html

6. Flask documentation. https://flask.palletsprojects.com/

7. Python Software Foundation. “sqlite3 — DB-API 2.0 interface for SQLite databases.” https://docs.python.org/3/library/sqlite3.html

Signature of the Team Member(s): _____

Date: _____